

* AOP of Micro-ESB

* In case error

1. Reload Cache

@Component

```
public record Startup(ResponseMappingRepository responseMappingRepository) {
```

```
    @Bean
```

```
    private void init() {
```

```
        try {
```

```
            List<ResponseMapping> responseMappingList = responseMappingRepository.findByStatusAndServiceCodeIn(
                GenConstants.ACT, new String[]{GenConstants.GEN, GenConstants.MESB});
```

```
            if (ResponseMappingCache.reloadAsList(responseMappingList) == 0) {
                throw new Exception("Responses mapping not found.");
            }
```

```
            // other cache here...
```

```
        } catch (Exception e) {
            e.printStackTrace();
        }
```

```
    }
```

```
}
```

2. Prepare cache

@Component

@Slf4j

```
public final class ResponseMappingCache {
```

```
    private static final String KEY = "microESBResponseMappingCacheKey";
```

```
    private static HashOperations<String, String, ResponseMapping> HASH_OPERATIONS;
```

```
    private final RedisTemplate<String, Object> redisTemplate;
```

```
    public ResponseMappingCache(
        RedisTemplate<String, Object> redisTemplate) {
        this.redisTemplate = redisTemplate;
    }
```

```
    @PostConstruct
```

```
    private void init() {
        HASH_OPERATIONS = redisTemplate.opsForHash();
    }
```

```
    public static int reloadAsList(List<ResponseMapping> list) {
        for (ResponseMapping item : list) {
            HASH_OPERATIONS.put(KEY, item.getSourceMsgCode(), item);
        }
        log.info("Reloaded response mapping to cache {} objects.", list.size());
        return list.size();
    }
```

```
    public static ResponseMapping getObject(String code) {
        return HASH_OPERATIONS.get(KEY, code);
    }
```

```
    public static void reloadAsObject(ResponseMapping item) {
        if (null != item) {
            HASH_OPERATIONS.put(KEY, item.getSourceMsgCode(), item);
            log.info("Reloaded response mapping to cache 1 object.");
        } else {
            log.info("Reloaded response mapping to cache 0 object.");
        }
    }
```

```
}
```

3. Prepare error response

```
public class AppException extends RuntimeException {
```

```
    private String sourceMsgCode;
```

```
    private HttpStatus httpStatus;
```

```
    private String[] params;
```

```
    public ResponseMapping getResponseMapping() {
        ResponseMapping responseMapping = ResponseMappingCache.getObject(sourceMsgCode);
        if (null == responseMapping) {
            responseMapping = ResponseMappingCache.getObject(MsgConstants.GEN_000);
        }
        this.httpStatus = HttpStatus.valueOf(responseMapping.getHttpStatus());
    }
```

```

//replace given params into response mapping msg
if (null != params) {
    for (int i = 0; i < params.length; i++) {
        responseMapping.setMsg(responseMapping.getMsg().replaceAll("\\{" + "p" + (i + 1) + "}", params[i]));
    }
}

return responseMapping;
}

public HttpStatus getHttpStatus() {
    if (this.httpStatus == null) {
        this.httpStatus = HttpStatus.BAD_REQUEST;
    }
    return httpStatus;
}

public AppException(String sourceMsgCode, String msg) {
    super(msg);
    this.sourceMsgCode = sourceMsgCode;
}

public AppException(String sourceMsgCode, String msg, String... params) {
    super(msg);
    this.sourceMsgCode = sourceMsgCode;
    this.params = params;
}
}
}

```

4. When using

```

public static String decryptRSA(String encryptedData, String privateKeyFilePath, String transformation) {
    try {
        FileInputStream fis = new FileInputStream(privateKeyFilePath);
        byte[] byteKeyFromFile = new byte[fis.available()];
        int bytesRead = fis.read(byteKeyFromFile);
        fis.close();

        if (bytesRead == -1) {
            // Handle case where no bytes were read
            throw new IOException("No bytes read from the file");
        }

        PKCS8EncodedKeySpec keySpec = new PKCS8EncodedKeySpec(byteKeyFromFile);
        KeyFactory factory = KeyFactory.getInstance("RSA");
        PrivateKey priKey = factory.generatePrivate(keySpec);
        Cipher c2 = Cipher.getInstance(transformation);
        c2.init(Cipher.DECRYPT_MODE, priKey);
        return new String(c2.doFinal(Base64.decodeBase64(encryptedData)));
    } catch (Exception ex) {
        // using AppException
        throw new AppException(MsgConstants.GEN_000, "Error while reading decrypted data");
    }
}

```

* Controller AOP

1. Prepare AOP

@Aspect

@Slf4j

@Component

public class ControllerAop {

@Around("execution(* com.plb.microesb.controller.*.*(..))")

public Object flow(ProceedingJoinPoint joinPoint) throws Throwable {

// check header pass correlation or not

if (null == joinPoint.getArgs()[1]) {

MDC.put("TRACE_ID", UUID.randomUUID().toString());

} else {

MDC.put("TRACE_ID", joinPoint.getArgs()[1].toString());

}

MDC.put("APP_CODE", joinPoint.getArgs()[0].toString());

ResponsePayload res;

try {

try {

log.info("[INT] : API : {}, REQ : {}", joinPoint.getSignature().toShortString(), joinPoint.getArgs()[2]);

// execute controller

Object result = joinPoint.proceed();

log.info("[END] : API : {}, RES : {}", joinPoint.getSignature().toShortString(), result);

return result;

} catch (AppException e) {

// throw app exception

res = new ResponsePayload();

res.prepareError(e.getResponseMapping());

log.error("[END] : API : {}, RES : {}", joinPoint.getSignature().toShortString(), res);

e.printStackTrace();

return new ResponseEntity<>(res, e.getHttpStatus());

} catch (Exception e) {

// throw other exception

try {

throw new AppException(MsgConstants.INT_SVR_ERR, "Internal server error");

} catch (AppException e1) {

res = new ResponsePayload();

res.prepareError(e1.getResponseMapping());

log.error("[END] : API : {}, RES : {}", joinPoint.getSignature().toShortString(), res);

e1.printStackTrace();

return new ResponseEntity<>(res, e1.getHttpStatus());

}

}

} catch (Exception e) {

// unexpected error

res = new ResponsePayload();

res.prepareUnexpectedError();

log.error("[END] : API : {}, RES : {}", joinPoint.getSignature().toShortString(), res);

e.printStackTrace();

return new ResponseEntity<>(res, HttpStatus.OK);

}

}

}

2. Prepare data for response

@JsonNaming(PropertyNamingStrategies.SnakeCaseStrategy.class)

@Data

public class ResponsePayload<T> {

private String code;

private String message;

private String correlationId;

private T data;

public ResponsePayload() {

this.code = "SUC-000";

this.message = "Successfully.";

this.correlationId = MDC.get("TRACE_ID");

this.data = null;

}

public ResponsePayload(T data) {

this.code = "SUC-000";

this.message = "Successfully.";

this.correlationId = MDC.get("TRACE_ID");

this.data = data;

}

```

public void prepareError(ResponseMapping response) {
    this.code = response.getTargetMsgCode();
    this.message = response.getMsg();
    this.correlationId = MDC.get("TRACE_ID");
    this.data = null;
}

public void prepareUnexpectedError() {
    this.code = "ERR-999";
    this.message = "Unexpected error.";
    this.correlationId = MDC.get("TRACE_ID");
    this.data = null;
}
}

3. When using
@PostMapping("/v1/metfone/decryption")
public ResponseEntity<ResponsePayload<Object>> decrypt(
    @RequestHeader(value = GenConstants.APP_CODE) String appCode,
    @RequestHeader(value = GenConstants.CORRELATION_ID, required = false) String correlationId,
    @RequestBody MetfoneDecryptionReq req) {
    MetfoneDecryptionRes res = metfoneService.getDecryptedData(req);
    // When set data to ResponsePayload
    return new ResponseEntity<>(new ResponsePayload<>(res), HttpStatus.OK);
}

```

* AOP of Merchant Project

```
@Aspect
@Slf4j
@Component
public class LogAop {

    @Around("within(com.plb.ib.controller..*)")
    public Object setLogTracing(ProceedingJoinPoint joinPoint) throws Throwable {
        MDC.put("TRACE_ID", UUID.randomUUID().toString());
        Object result = joinPoint.proceed();
        return result;
    }

}
```

* AOP of ISO Gateway

```
@Aspect
@Component
@Slf4j
public class ApiAop extends AppProperties {

    @Around("execution(* com.plb.isogateway.controller.*.*(..))")
    public Object flow(ProceedingJoinPoint joinPoint) throws Throwable {
        try {
            Object[] args = joinPoint.getArgs();

            // Safe null and bounds checks
            String mdcCorrelationId = (args.length > 0 && args[0] != null) ? args[0].toString() : correlationId;
            String mdcAppCode = (args.length > 1 && args[1] != null) ? args[1].toString() : appCode;

            // Add context to MDC
            MDC.put(Constant.CORRELATION_ID, mdcCorrelationId);
            MDC.put(Constant.APP_CODE, mdcAppCode);

            // Proceed with method execution
            return joinPoint.proceed();
        } catch (Exception ex) {
            log.error("Error during API execution: {}", ex.getMessage(), ex);
            throw ex;
        } finally {
            // Clear MDC to prevent memory leaks
            MDC.clear();
        }
    }

}
```

* Sample AOP

```
@Aspect
@Component
@Slf4j
public class LoggingAop {

    @Around("execution(* com.plb.camelmi.*.*.*.*(..))")
    public Object logAround(ProceedingJoinPoint joinPoint) throws Throwable {
        if (MDC.get(CommonConstants.TRACE_ID) == null) {
            MDC.put(CommonConstants.TRACE_ID, UUID.randomUUID().toString());
        }
        long startTime = System.currentTimeMillis();
        String methodName = joinPoint.getSignature().toShortString();
        Object[] args = joinPoint.getArgs();

        log.info("➡ Entering {} with arguments = {}", methodName, Arrays.toString(args));
        Object result;
        try {
            result = joinPoint.proceed();
            long duration = System.currentTimeMillis() - startTime;
            log.info("✅ Exiting {} with result = {} (Execution time: {} ms)", methodName, result, duration);
            return result;
        } catch (Throwable ex) {
            log.error("❌ Exception in {} with message = {}", methodName, ex.getMessage(), ex);
            throw ex;
        }
    }

}
```

◆ Example AOP 1

1. Create a Service Class

```
@Service
public class MyService {
    public void process() {
        System.out.println(">>> Business logic running...");
    }
}
```

2. Create an Aspect Class

```
@Aspect
@Component
public class LoggingAspect {

    // This advice will run BEFORE any method in MyService
    @Before("execution(* com.example.demo.MyService.*(..))")
    public void logBefore(JoinPoint joinPoint) {
        System.out.println(">>> [LOG] Before method: " + joinPoint.getSignature().getName());
    }

    // This advice will run AFTER the method finishes (success or exception)
    @After("execution(* com.example.demo.MyService.*(..))")
    public void logAfter(JoinPoint joinPoint) {
        System.out.println(">>> [LOG] After method: " + joinPoint.getSignature().getName());
    }
}
```

◆ Output Example

```
>>> [LOG] Before method: process
>>> Business logic running...
>>> [LOG] After method: process
```


◆ Example AOP 2

```
@Around("execution(* com.plb.microesb.controller.*.*(..))") // Apply this advice to all methods inside com.plb.microesb.controller
package
public Object flow(ProceedingJoinPoint joinPoint) throws Throwable {

    // Step 1: Extract correlation ID and APP_CODE from method arguments
    // Assume joinPoint.getArgs()[0] = APP_CODE, joinPoint.getArgs()[1] = correlation ID (TRACE_ID)
    if (null == joinPoint.getArgs()[1]) {
        // If correlation ID is null, generate a new UUID as TRACE_ID
        MDC.put("TRACE_ID", UUID.randomUUID().toString());
    } else {
        // Otherwise, use the provided correlation ID
        MDC.put("TRACE_ID", joinPoint.getArgs()[1].toString());
    }

    // Put APP_CODE into MDC for logging purposes
    MDC.put("APP_CODE", joinPoint.getArgs()[0].toString());

    // Declare response object
    ResponsePayload res;

    try {
        try {
            // Step 2: Log the incoming request
            log.info("[INT] : API : {}, REQ : {}",
                joinPoint.getSignature().toShortString(), // Log method name
                joinPoint.getArgs()[2]); // Log request body (3rd argument)

            // Step 3: Execute the actual controller method
            Object result = joinPoint.proceed();

            // Step 4: Log the response after method execution
            log.info("[END] : API : {}, RES : {}",
                joinPoint.getSignature().toShortString(),
                result);

            // Step 5: Return the result back to client
            return result;

        } catch (Exception e) {
            // Handle exception if needed here (optional inner catch block)
            throw e;
        }
    } finally {
        // Step 6: Clear MDC to avoid data leak across threads
        MDC.clear();
    }
}
```

What we should understand before learning AOP in Spring Boot.

🔍 What is AOP (Aspect-Oriented Programming)?

AOP is a programming technique in Spring that helps you separate common logic (like logging, security, or transactions) from your business logic.

🔍 What is MDC?

MDC stands for Mapped Diagnostic Context.

It's a feature provided by logging frameworks like SLF4J, Logback, or Log4j to store contextual information (like TRACE_ID, APP_CODE, USER_ID, etc.) in a thread-local map during logging.

- ◆ AOP Annotations

➡ Marks the class as an **Aspect**, so it can contain AOP logic (e.g., @Around, @Before).

@Aspect

- ◆ Pointcut Expression

➡ Run the method **around all methods** in any class under com.plb.microesb.controller.

@Around("execution(* com.plb.microesb.controller.*.*(..))")

- ◆ Check if argument is null

➡ Check if the **second method argument** (index 1) is null. Often used to decide whether to generate a new trace ID.

if (null == joinPoint.getArgs()[1])

- ◆ Extracting method arguments

➡ Get all method arguments as an array.

Object[] args = joinPoint.getArgs();

➡ Get the **number of arguments** passed to the method.

args.length

➡ Get the **first argument**.

args[0]

- ◆ MDC (Mapped Diagnostic Context)

➡ Generate and store a random trace ID in the logging context (MDC).

MDC.put("TRACE_ID", UUID.randomUUID().toString());

➡ Retrieve the current trace ID from MDC.

MDC.get("TRACE_ID");

- ◆ Method Metadata

➡ Get a **short description** of the method (e.g., ClassName.methodName(..)).

joinPoint.getSignature().toShortString()

- ◆ Accessing specific arguments

➡ Convert the **first argument** to a String (e.g., for logging APP_CODE).

joinPoint.getArgs()[0].toString()

➡ Access the **third argument**, usually the **request body** or payload.

joinPoint.getArgs()[2]

- ◆ Proceed with actual method execution

➡ Call the **original method** (e.g., the controller) and capture the result.

Object result = joinPoint.proceed();

- ◆ Return HTTP response

➡ Return an HTTP response with a body (res) and a custom HTTP status code (from exception e).

return new ResponseEntity<>(res, e.getHttpStatus());

- ◆ Print exception to console

➡ Print the full error stack trace (for debugging, but better to use logging in real apps).

```
e.printStackTrace();
```

- ◆ Proceed and return directly

➡ Execute the method and **return the result immediately** without custom logic.

```
return joinPoint.proceed();
```

- ◆ Throw an exception

➡ Throw the caught or created exception (so it can be handled by Spring's global handler).

```
throw ex;
```

- ◆ Clean up logging context

➡ Remove all stored values from the logging context — very important to avoid leaking TRACE_ID into the next request/thread.

```
MDC.clear();
```

✅ Top Benefits of Using AOP with Examples

1. 🔍 Centralized Logging

Without AOP: You add logging in every controller/service method manually.

With AOP: You write it once and apply it to all methods automatically.

```
@Around("execution(* com.bank.api..*(..))")
public Object logRequestResponse(ProceedingJoinPoint jp) throws Throwable {
    log.info("START: {} - Args: {}", jp.getSignature(), Arrays.toString(jp.getArgs()));
    Object result = jp.proceed();
    log.info("END: {} - Result: {}", jp.getSignature(), result);
    return result;
}
```

✅ **Benefit:** Clean code, less duplication, easier debugging.

2. ⌚ Track API Performance

Measure how long APIs or DB calls take.

```
@Around("execution(* com.bank.service..*(..))")
public Object monitorTime(ProceedingJoinPoint jp) throws Throwable {
    long start = System.currentTimeMillis();
    Object result = jp.proceed();
    log.info("Execution Time for {}: {}ms", jp.getSignature(), System.currentTimeMillis() - start);
    return result;
}
```

✅ **Benefit:** Identify slow services or bottlenecks.

3. 📄 Audit Trail Logging

Capture who changed what and when (especially useful in core banking, CMS, or LOS systems).

```
@Before("execution(* com.bank.service.*.update*(..))")
public void audit(JoinPoint jp) {
    String user = SecurityContextHolder.getContext().getAuthentication().getName();
    log.info("User '{}' invoked {} with {}", user, jp.getSignature(), Arrays.toString(jp.getArgs()));
}
```

✅ **Benefit:** Support compliance, traceability, and accountability.

4. 🛡️ Authorization Check

Instead of putting `if (!hasPermission)` in every method...

```
@Before("execution(* com.bank.service.AdminService.*(..))")
public void checkPermission(JoinPoint jp) {
    String role = getCurrentUserRole();
    if (!role.equals("ADMIN")) {
        throw new AccessDeniedException("You are not allowed to access: " + jp.getSignature());
    }
}
```

✅ **Benefit:** Clean security logic, apply role checks globally.

5. 🛠️ Input Validation or Sanitization

Automatically sanitize or validate inputs before entering the method:

```
@Before("execution(* com.bank.controller.*(..))")
public void validateInputs(JoinPoint jp) {
    for (Object arg : jp.getArgs()) {
        if (arg instanceof String && ((String) arg).contains("DROP")) {
            throw new RuntimeException("Dangerous input detected");
        }
    }
}
```

✅ **Benefit:** Protect against injection attacks or malformed input.

6. 🔄 Retry Failed Operation

AOP can be used to retry a failed method (e.g., API call, DB connection):

```
@Around("execution(* com.bank.integration.ExternalApiClient.*(..))")
public Object retry(ProceedingJoinPoint jp) throws Throwable {
    int attempts = 3;
    while (attempts-- > 0) {
        try {
            return jp.proceed();
        } catch (Exception ex) {
            if (attempts == 0) throw ex;
            log.warn("Retrying {} due to {}", jp.getSignature(), ex.getMessage());
        }
    }
    return null;
}
```

✅ **Benefit:** Improve fault-tolerance.

7. 📦 Wrap All Responses with Standard Format

Ensure all controllers return a consistent structure:

```
@Around("execution(* com.bank.api.controller.*(..))")
public Object wrapResponse(ProceedingJoinPoint jp) throws Throwable {
    Object data = jp.proceed();
    return new StandardResponse("SUC-000", "SUCCESS", data);
}
```

✅ **Benefit:** Unified API response handling.

8. 📝 Logging Request & Response

Use `@Around` to log all REST API inputs and outputs automatically.

```
@Around("execution(* com.bank.api.controller.*(..))")
public Object logRequestResponse(ProceedingJoinPoint joinPoint) throws Throwable {
    log.info("REQ: {} - {}", joinPoint.getSignature(), Arrays.toString(joinPoint.getArgs()));
    Object result = joinPoint.proceed();
    log.info("RES: {} - {}", joinPoint.getSignature(), result);
    return result;
}
```

✔ Benefit: Helps debug easily without adding logs in every method.

✔ 9. Execution Time Monitoring (Performance)

Track how long a method takes to execute.

```
@Around("execution(* com.bank.service..*(..))")
public Object measureExecutionTime(ProceedingJoinPoint joinPoint) throws Throwable {
    long start = System.currentTimeMillis();
    Object result = joinPoint.proceed();
    long duration = System.currentTimeMillis() - start;
    log.info("Execution time of {}: {} ms", joinPoint.getSignature(), duration);
    return result;
}
```

✔ Benefit: Detect slow services or DB calls.

✔ 10. Global Exception Catching

You can use AOP to catch and handle uncaught exceptions cleanly (not a replacement for @ControllerAdvice, but useful in service layer):

```
@Around("execution(* com.bank.service..*(..))")
public Object catchException(ProceedingJoinPoint joinPoint) {
    try {
        return joinPoint.proceed();
    } catch (Exception ex) {
        log.error("Error in method: {}", joinPoint.getSignature(), ex);
        throw ex; // rethrow to be handled globally
    }
}
```

✔ 11. Audit Logging

Track who made changes to important entities or systems.

```
@Before("execution(* com.bank.service.AccountService.update*(..))")
public void audit(JoinPoint joinPoint) {
    String user = SecurityContextHolder.getContext().getAuthentication().getName();
    log.info("User {} is calling {}", user, joinPoint.getSignature());
}
```

✔ Benefit: You get security and audit compliance for free.

✔ 12. Validate Required Headers / Tokens

Before controller is called, check if a required header is passed.

```
@Before("execution(* com.bank.controller..*(..))")
public void validateHeaders(JoinPoint joinPoint) {
    // You can access HttpServletRequest from RequestContextHolder
    ServletRequestAttributes attr = (ServletRequestAttributes) RequestContextHolder.currentRequestAttributes();
    HttpServletRequest request = attr.getRequest();

    String token = request.getHeader("X-Auth-Token");
    if (token == null) {
        throw new RuntimeException("Missing X-Auth-Token");
    }
}
```

✔ Benefit: Centralized API validation.